JavaScript

AGENDA

1. What is JavaScript?
2. Syntax
3. Variables and Scope
4. Data Types and conversion
5. Functions
6. Conditionals
7. Loops
8. Arrays
9. Objects
10. Control flow and Operators
11. Best practices
12. Mini-exercise + QA

1. What is JavaScript?

JAVASCRIPT

- JavaScript (JS) is a high-level, dynamic, prototype-based programming language standardized as ECMAScript.

Where does JS run?

1. Browsers (Frontend).
2. Backend (NodeJs, Deno, etc)
3. Serverless (Cloud), AWS Lambda, GCP/Azure functions
4. Desktop
5. Mobile, etc

2. Syntax

SYNTAX

- Statements end with ; (optional, but consistency helps)
- Case-sensitive identifiers
- Comments: // and /* ... */
- Whitespace is mostly cosmetic
- "use strict" enables stricter checks

Case-Sensitive JS

Cat ≠ cat

```js
let name = "Alice";
let Name = "Bob";
let NAME = "Charlie";

console.log(name); // "Alice"
console.log(Name); // "Bob"
console.log(NAME); // "Charlie"
```

```js
let sum = 1 + 2;
console.log(sum);
```

js

```js
let   sum=1+2;console.log(sum);
```

```
let
sum
=
1
+
2
;
console
.
log
(
sum
)
;
```

3. Variables and Scope

VARIABLES AND SCOPE

- **const:** block-scoped, no reassignment (default choice)
- **let:** block-scoped, reassignment allowed
- **var:** function-scoped; hoisted (avoid in modern code)

Keyword	const	let	var
global scope	✗	✗	✓
function scope	✓	✓	✓
block scope	✓	✓	✗
can be reassigned	✗	✓	✓

```
$(document).ready(function () {
    {
        var myVar = "bla";
        console.log(myVar);
    }
    console.log(myVar);

    {
        let myLet = "blub";
        console.log(myLet);
    }
    console.log(myLet);

    {
        const myConst = "blib";
        myConst = "blob";
        console.log(myConst);
    }
    console.log(myConst);
});
```

4. Data types and conversion

DATA TYPES

1. **Primitive Data Types:** Number, BigInt, String, Boolean, Undefined, Null Symbol.
2. **Non-Primitive (Reference) Types:** Object, Array, Function, Date, RegExp, Map, Set, WeakMap, WekSet

Primitive		Reference	
null	let example = null;	**Object**	let example = {hello: "world"};
undefined	let example = undefined;	**Function**	let example = () => 2 + 2;
Boolean	let example = true;		
Number	let example = 33;		
String	let example = "hello";		
BigInt	let example = 33n;		
Symbol	let example = Symbol("hello");		

CONVERSION

1. Implicit vs explicit conversion
2. Falsy values: false, 0, -0, "", null, undefined, NaN
3. Prefer strict equality === over ==

```
if (!false) console.log("false is falsy");
if (!0) console.log("0 is falsy");
if (!-0) console.log("-0 is falsy");
if (!"") console.log('"" (empty string) is falsy');
if (!null) console.log("null is falsy");
if (!undefined) console.log("undefined is falsy");
if (!NaN) console.log("NaN is falsy");
```

```
0 == false       // true (both become 0)
0 === false      // false (different types)
null == undefined // true (special case)
null === undefined // false
" " == 0         // true (empty string becomes 0)
" " === 0        // false
```

Explicit vs Implicit Conversion

- **Explicit**: using constructors or functions

```js
Number("42");  // 42
Boolean(0);    // false
String(true);  // "true"
```

- **Implicit**: JavaScript does it automatically

```js
"5" == 5; // true (string converted to number)
```

5. Functions

FUNCTIONS

- Declarations are hoisted
- Arrow functions are concise, lexical this
- Defaults & rest params

```
const user = {
  name: "Alice",
  regularFunc: function() {
    console.log(this.name); // 'Alice'
  },
  arrowFunc: () => {
    console.log(this.name); // undefined (inherits from global scope)
  }
};

user.regularFunc(); // ✅ "Alice"
user.arrowFunc();   // ❌ undefined
```

✅ Example — works (hoisted declaration)

```js
greet(); // ✅ Works because the declaration is hoisted

function greet() {
  console.log("Hello!");
}
```

❌ Example — doesn't work (function expression)

```js
sayHi(); // ❌ ReferenceError: Cannot access 'sayHi' before initialization

const sayHi = function() {
  console.log("Hi!");
};
```

FUNCTIONS (DEFAULT AND REST PARAMS)

- You can assign default values to function parameters — they're used when an argument is missing or undefined.
- The rest operator collects all remaining arguments into an array. This is useful when you don't know how many arguments will be passed.

```
function greet(name = "Guest") {
  console.log("Hello " + name);
}

greet("Jose");  // Hello Jose
greet();        // Hello Guest
```

```
function sum(...nums) {
  return nums.reduce((total, n) => total + n, 0);
}

console.log(sum(1, 2, 3, 4)); // 10
```

6. CONDITIONALS

CONDITIONALS

- if / else, switch, ternary
- Optional chaining ?., nullish coalescing ??
- OR operator || : returns the first truthy value or the last value if none are truthy.
- ?? is similar to ||, but only considers null and undefined as “missing”

```
let age = 18;

if (age >= 18) {
  console.log("Adult");
} else if (age >= 13) {
  console.log("Teenager");
} else {
  console.log("Child");
}
```

```
let age = 18;
let category;

switch (true) {
  case age >= 18:
    category = "Adult";
    break;
  case age >= 13:
    category = "Teenager";
    break;
  default:
    category = "Child";
}

console.log(category);
```

```
const role = "admin";
const label = role === "admin" ? "A" : "U";

const user = { profile: null };
const city = user.profile?.address?.city ?? "Unknown";
```

CONDITIONALS

```
let age = 20;
let message = age >= 18 ? "Welcome!" : "Too young!";
console.log(message); // "Welcome!"
```

```
0 || 100;        // 100
"" || "text";    // "text"
false || true;   // true
```

```
let count = 0;
let result = count ?? 10;
console.log(result); // 0 ✅
```

```
let user = {
  name: "",
  age: null
};

let displayName = user.name || "Anonymous";
let displayAge = user.age ?? "Unknown";

console.log(displayName); // "Anonymous"
console.log(displayAge);  // "Unknown"
```

7. LOOPS

LOOPS

- for, while, do..while
- for...of → values (arrays/iterables)
- for...in → keys (objects; avoid for arrays)
- break / continue

```
// FOR
for (let i = 0; i < 3; i++) {
  console.log("for:", i);
}
```

```
// WHILE
let j = 0;
while (j < 3) {
  console.log("while:", j);
  j++;
}
```

```
// DO...WHILE
let k = 0;
do {
  console.log("do...while:", k);
  k++;
} while (k < 3);
```

```
// FOR...OF → values (arrays / iterables)
const nums = [10, 20, 30];
for (const n of nums) {
  console.log("for...of value:", n);
}
```

```
// FOR...IN → keys (object properties)
const user = { name: "Alice", age: 25 };
for (const key in user) {
  console.log("for...in key:", key, "value:", user[key]);
}
```

```
// BREAK / CONTINUE
for (let i = 0; i < 5; i++) {
  if (i === 2) continue; // skip 2
  if (i === 4) break;    // stop loop
  console.log("break/continue:", i);
}
```

LOOPS

```
const nums = [1, 2, 3, 4, 5];

// forEach → just loops (no return)
nums.forEach(n => {
  console.log("forEach:", n); // prints each number
});

// map → transforms values and returns a new array
const doubled = nums.map(n => n * 2);
console.log("map:", doubled); // [2, 4, 6, 8, 10]

// filter → keeps only values that match condition
const even = nums.filter(n => n % 2 === 0);
console.log("filter:", even); // [2, 4]

// combine map + filter
const squaredEven = nums
  .filter(n => n % 2 === 0)
  .map(n => n ** 2);
console.log("squaredEven:", squaredEven); // [4, 16]
```

8. ARRAYS

ARRAYS

- An array is a special type of object used to store multiple values in a single variable, ordered by index.

```
// Create an array
const numbers = [1, 2, 3, 4];

// Access elements
console.log(numbers[0]);        // First element
console.log(numbers.length);    // Size of array

// Add element
numbers.push(5);

// map() → transform
const doubled = numbers.map(n => n * 2);

// filter() → select
const evens = numbers.filter(n => n % 2 === 0);

// reduce() → aggregate
const sum = numbers.reduce(
  (acc, curr) => acc + curr,
  0
);

console.log(doubled);
console.log(evens);
console.log(sum);
```

OBJECTS

- Create with {} or “new”
- Dot vs bracket
- accessMethods & this
- Destructuring & shorthand
- Shallow copy with spread operator

```
const key = "fav-color";
const user = {
  name: "Ana",
  greet(){ return `Hi ${this.name}`; }
};
user.age = 28;
user[key] = "blue";

const { name, age = 0 } = user;   // destructuring with default
const clone = { ...user, role: "admin" }; // shallow copy
```

OBJECTS

```
// Define a constructor function
function Person(name, age) {
  this.name = name;
  this.age = age;
  this.greet = function() {
    console.log(`Hi, I'm ${this.name} and I'm ${this.age} years old.`);
  };
}

// Create an object using "new"
const person1 = new Person("Alice", 25);
const person2 = new Person("Bob", 30);

person1.greet(); // Hi, I'm Alice and I'm 25 years old.
person2.greet(); // Hi, I'm Bob and I'm 30 years old.
```

10. CONTROL FLOW AND OPERATORS

CONTROL FLOW

- Sequential Execution (Default Behavior)
- Conditional Control Flow
- Loops
- Control Statements Inside Loops
- Error Handling (Control Flow via Exceptions)
- Asynchronous Control Flow

```javascript
console.log("Step 1");
console.log("Step 2");
console.log("Step 3");
```

```javascript
try {
  const result = 10 / 0;
  console.log(result);
} catch (error) {
  console.log("An error occurred");
} finally {
  console.log("Execution finished");
}
```

```javascript
async function fetchData() {
  try {
    const response = await fetch("https://api.example.com/data");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log("Error fetching data");
  }
}

fetchData();
```

OPERATORS

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- Logical Operators
- Ternary Operator
- Type Operators
- Spread & Rest Operators
- Optional Chaining

```
const age = 18;

const message = age >= 18
  ? "Adult"
  : "Minor";

console.log(message);
```

```
console.log(typeof 42);        // "number"
console.log(typeof "Hello");   // "string"
console.log(typeof true);      // "boolean"
```

```
class User {}

const user = new User();

console.log(user instanceof User); // true
```

```
const arr1 = [1, 2];
const arr2 = [...arr1, 3, 4];

console.log(arr2); // [1, 2, 3, 4]
```

```
const user = {};

console.log(user.address?.city);
```

11. BEST PRACTICES

BEST PRACTICES

- Use const by default and let when needed (avoid var)
- Always use strict equality (=== and !==)
- Use meaningful and descriptive variable names
- Keep functions small and focused (Single Responsibility)
- Avoid deep nesting (use early returns)
- Handle errors properly with try/catch
- Use modern ES6+ features (arrow functions, spread, template literals)
- Keep consistent code style (ESLint & Prettier)
- Prefer immutable patterns (avoid mutating data)
- Write readable code over clever code

❌ Hard to read:

JavaScript

```javascript
if (user) {
  if (user.isActive) {
    if (user.role === "admin") {
      // ...
    }
  }
}
```

✔ Better (Early Return):

JavaScript

```javascript
if (!user || !user.isActive) return;
if (user.role !== "admin") return;
```

12. MINI-EXERCISE + QA

QUESTIONS

- **1)** Which of the following is a single-line comment in JavaScript?

  A. <!-- comment -->

  B. # comment

  C. // comment

  D. /* comment */

QUESTIONS

✅ **Answer:** C
➡️ **Explanation:** JavaScript uses // for single-line comments.
/* ... */ is for multi-line comments.

- **2)** What is the correct way to print text in JavaScript?

  A. print("Hello");

  B. echo("Hello");

  C. console.log("Hello");

  D. alert.log("Hello");

QUESTIONS

✅ **Answer:** C
➡️ **Explanation:** console.log() prints output to the browser's console.

- **3)** Which of these is **NOT** a valid JavaScript variable name?

  A. name

  B. _age

  C. 2value

  D. $price

QUESTIONS

✅ **Answer:** C
➡️ **Explanation:** Variable names **cannot start with a number**.

- **4)** What keyword declares a variable that **can change** later?

  A. const

  B. let

  C. final

  D. static

QUESTIONS

✅ **Answer:** B
➡️ **Explanation:** let (or var) allows reassignment. const does not.

- **5)** What does this print?

```
let x = 5;
x = 10;
console.log(x);
```

A. 5
B. 10
C. undefined
D. error

QUESTIONS

✅ **Answer:** B
➡️ **Explanation:** The variable x is reassigned to 10 before printing.

6) What will happen here?

```
const name = "Ana";
name = "Luis";
console.log(name);
```

A. Ana
B. Luis
C. undefined
D. Error

QUESTIONS

✅ **Answer:** D
➡️ **Explanation:** You cannot reassign a const variable.

7) Which of the following is **not** a JavaScript data type?

A. string
B. number
C. character
D. boolean

QUESTIONS

✅ **Answer:** C
➡️ **Explanation:** JavaScript does not have a separate char type; characters are strings of length 1.

8) What will this print?

```
let x = 10;
let y = "10";
if (x === y) {
  console.log("Equal");
} else {
  console.log("Not Equal");
}
```

A. Equal
B. Not Equal
C. undefined
D. Error

QUESTIONS

✅ **Answer:** B
➡️ **Explanation:** === checks type and value. One is number, the other string → Not equal

9) What happens here?

```
let car = {brand: "Toyota"};
car.model = "Corolla";
console.log(car);
```

A. {brand: "Toyota"}
B. {brand: "Toyota", model: "Corolla"}
C. {model: "Corolla"}
D. Error

QUESTIONS

✅ **Answer:** B
➡️ **Explanation:** You can add new properties to an object dynamically.

10) What does this print?

```
let obj = {x: 10, y: 20};
console.log("x" in obj);
```

A. true
B. false
C. "x"
D. Error

QUESTIONS

✅ **Answer:** A
➡️ **Explanation:** The in operator checks if a property exists in an object.

11) What does this print?

```
console.log(a);
var a = 10;
console.log(b);
let b = 20;
```

A. undefined then 20
B. 10 then 20
C. undefined ReferenceError
D. ReferenceError undefined

QUESTIONS

✅ **Answer:** C
➡️ **Explanation:**
var declarations are hoisted and initialized as undefined.
let declarations are hoisted but **not initialized**, so accessing b before its declaration throws a **ReferenceError** (TDZ — Temporal Dead Zone).

12) What does this print?

```
console.log([] == 0);
console.log("0" == []);
console.log([] == false);
```

A. true true true
B. true false false
C. true true false
D. true false true

QUESTIONS

✅ **Answer:** D

➡️ **Explanation:**

1. `[] == 0` → object vs number
   - `[]` → ToPrimitive → `""` (arrays stringify to empty string)
   - `"" == 0` → ToNumber("") → `0` → `0 == 0` → **true**
2. `"0" == []` → string vs object
   - `[]` → ToPrimitive → `""`
   - `"0" == ""` → both are strings, compare directly → **false**
3. `[] == false` → object vs boolean
   - `false` → ToNumber(false) → `0`
   - Now `[] == 0` (from #1) → **true**

13) What does this print?

```
const obj1 = { value: 10 };
const obj2 = obj1;
obj2.value = 99;
console.log(obj1.value);
```

A. 10

B. 99

C. undefined

D. Error

QUESTIONS

✅ **Answer:** B
➡️ **Explanation:**
Objects are **reference types**. obj2 and obj1 point to the same object in memory, so changing one affects the other.

14) What does this print?

```
let count = 5;
function test() {
  console.log(count);
  let count = 10;
}
test();
```

A. 5
B. 10
C. undefined
D. ReferenceError

QUESTIONS

15) What does this print?

A. 5, 20, 10

B. undefined, 20, 10

C. undefined, 10, 20

D. 5, 10, 20

```
var x = 5;

function test() {
  console.log(x);
  var x = 10;
  if (true) {
    let x = 20;
    console.log(x);
  }
  console.log(x);
}

test();
```

✅ **Answer:** D

➡️ **Explanation:**

Inside the function, the local variable count (declared with let) **shadows** the outer one.

It exists but is **in the Temporal Dead Zone** until its declaration, so accessing it before let count = 10 causes a **ReferenceError**.

QUESTIONS

✅ **Answer: B**
➡️ **Explanation:**
At the start of the function, JavaScript **hoists** var x to the top of the test() function, but **without its value**. So inside test(), x exists but is initially undefined.
First console.log(x) prints undefined (the local x, not the global one).
The let x = 20 inside the if block creates a **new block-scoped variable** that **shadows** the var x. So the second log prints 20.
After leaving the block, we go back to the function's var x, now assigned 10, so the last log prints 10.

Output:
undefined
20
10

THANKS